

Rapport de Projet - SpotifYnov – NoSQL

Prepared By : KHADIJA IBISK

Introduction:

Ce rapport présente la mise en œuvre de l'architecture backend de Spotify. Pour répondre aux besoins de performance d'une plateforme de streaming moderne, j'ai mis en place une architecture Polyglot Persistence utilisant quatre technologies NoSQL (MongoDB, Cassandra, Redis et Neo4j), chacune répondant à une problématique spécifique : catalogue, analytique, performance et recommandations.

```
PS C:\Users\DELL> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS                                                
```

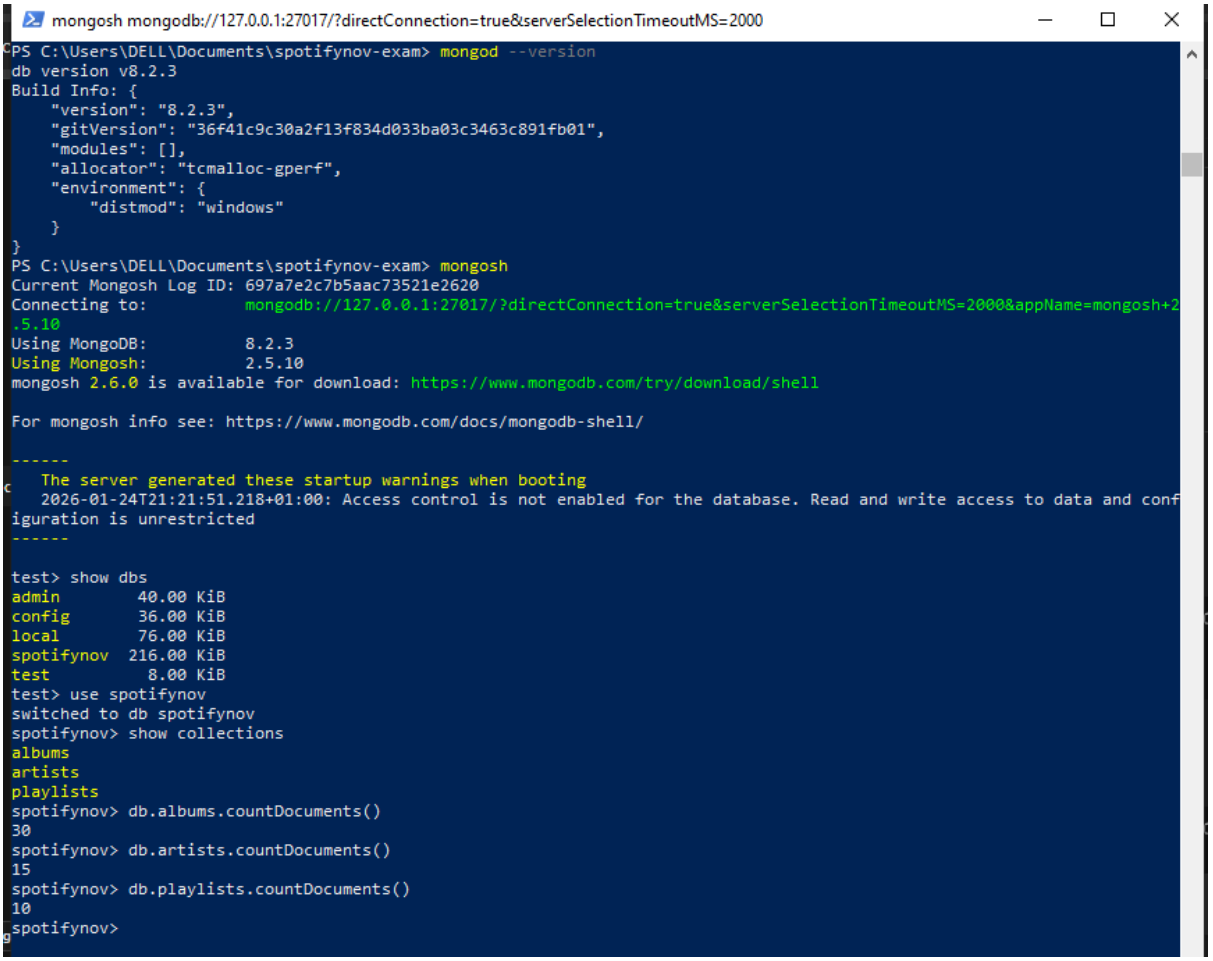
1.1 Modélisation et Données

J'ai opté pour un modèle orienté document permettant de regrouper les informations liées..

- **Embarquement (Embedding) :** Les pistes (tracks) sont directement intégrées dans les documents albums pour minimiser les jointures lors de la consultation d'un album.
- **Référencement (Linking) :** les relations entre artistes et albums se font par nom ou ID afin d'éviter une duplication excessive.

1.2 Implémentation

- **Collections :** `artists`, `albums`, `playlists`.
- **Agrégations :** Mise en place de pipelines pour calculer les statistiques par genre et par pays (voir `mongodb_aggregations.js`).
- **Indexation :** Création d'index textuels pour la recherche rapide d'artistes.



PARTIE 2 : CASSANDRA - HISTORIQUE ET ANALYTICS

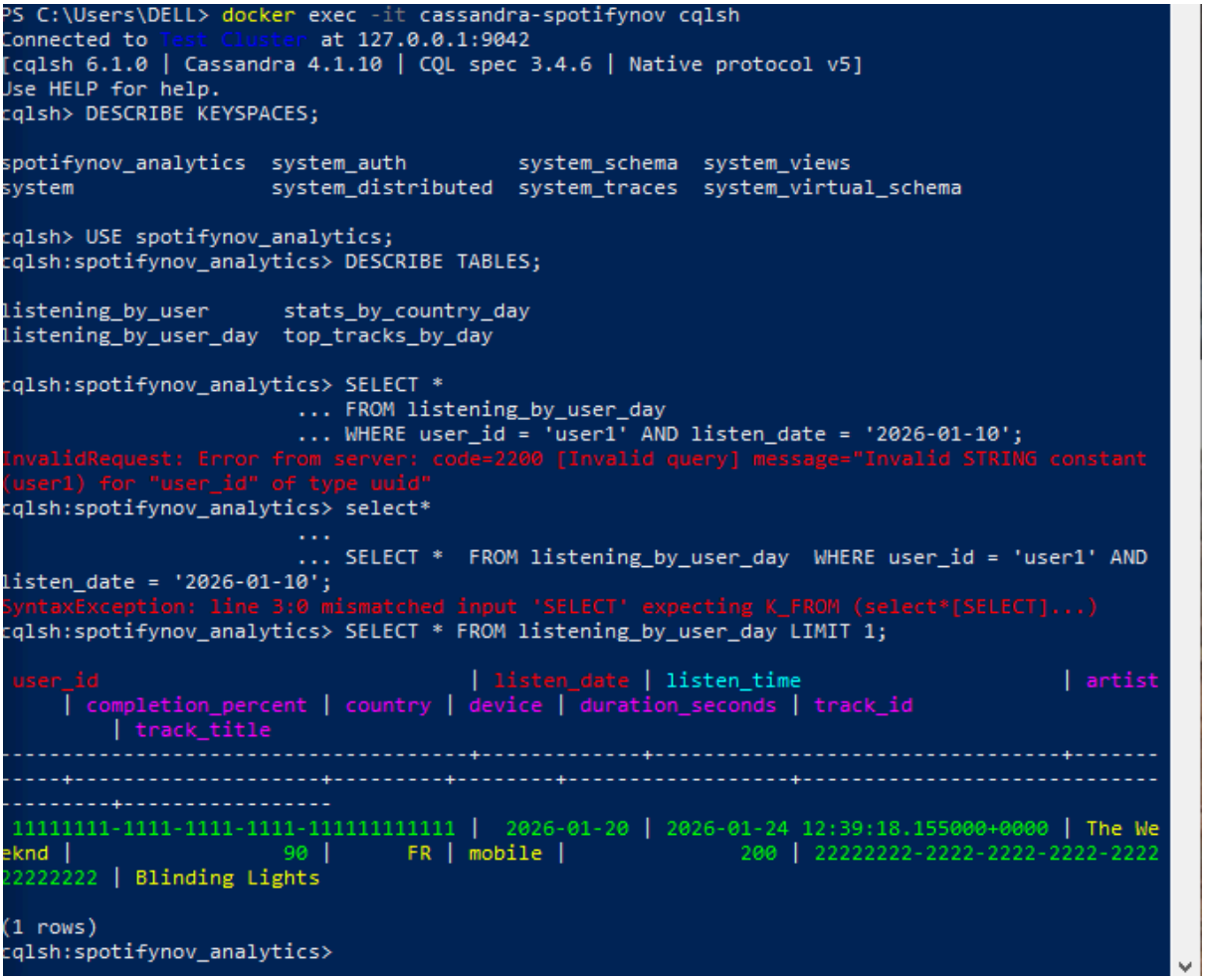
2.1 Modélisation Time-Series

La modélisation a été faite selon l'approche **"Query-First"**.

- **Partitionnement :** La table `listening_by_user_day` utilise une clé de partition composite `(user_id, listen_date)` pour regrouper les écoutes par utilisateur et par jour.
- **Clustering :** Les données sont triées par `listen_time DESC` pour afficher les dernières écoutes en priorité.

2.2 Statistiques et Compteurs

L'utilisation du type COUNTER dans les tables `stats_by_country_day` et `top_tracks_by_day` permet une mise à jour atomique et rapide des compteurs globaux sans lecture préalable.



PARTIE 3 : REDIS - SESSIONS ET CACHE

Objectif : Optimiser la latence et gérer l'état temps réel.

```
PS C:\Users\DELL> docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED         STATUS          PORTS
                                                                                       NAMES
5fd721877e2e   redis           "docker-entrypoint.s…"   32 hours ago    Up 22 minutes   6379/tc
p                                                                                      mon-redis
693453ff53fe   neo4j:5         "tini -g -- /startup…"   7 days ago      Up 22 minutes   0.0.0.0
:7474->7474/tcp, [::]:7474->7474/tcp, 0.0.0.0:7687->7687/tcp, [::]:7687->7687/tcp   neo4j-spot
ifynov
1dafde7a5b60   redis:7         "docker-entrypoint.s…"   7 days ago      Up 22 minutes   0.0.0.0
:6379->6379/tcp, [::]:6379->6379/tcp                                                   redis-spot
ifynov
4b1d51373ae3   cassandra:4.1   "docker-entrypoint.s…"   7 days ago      Up 22 minutes   0.0.0.0
:9042->9042/tcp, [::]:9042->9042/tcp                                                   cassandra-
spotifynov
```

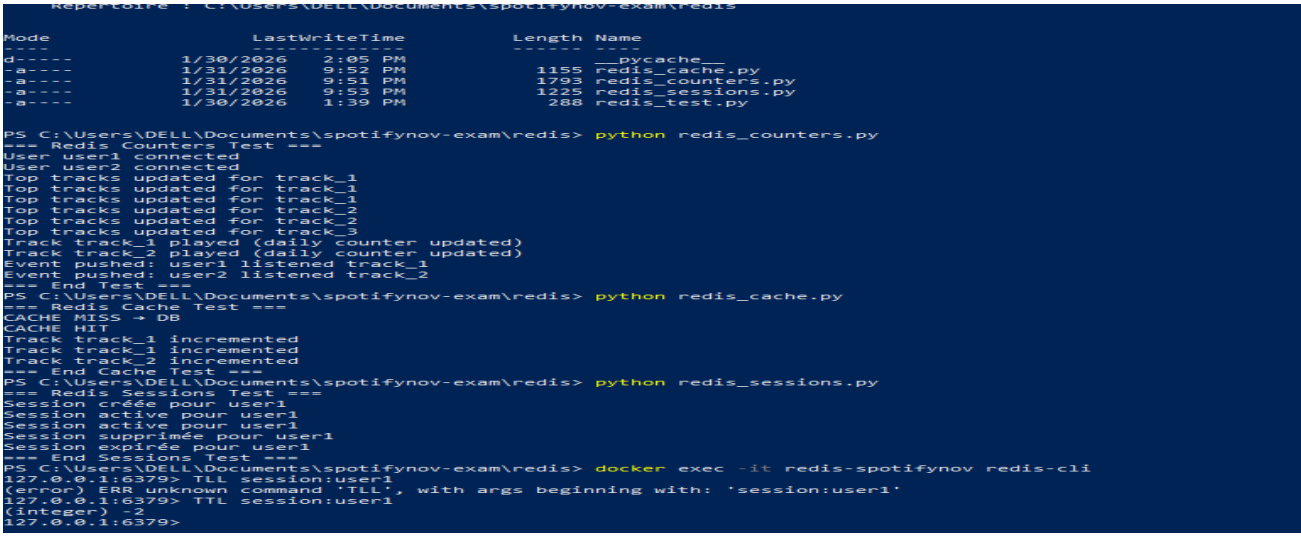
3.1 Stratégie de Cache

- **Pattern Cache-Aside :** Implémenté pour les profils d'artistes. L'application vérifie Redis avant d'interroger MongoDB, réduisant la charge sur la base principale (voir `redis_cache.py`).
- **TTL (Time To Live) :** Configuré à 60s pour garantir la fraîcheur des données.

```
PS C:\Users\DELL> docker exec -it redis-spotifynov redis-cli ping
PONG
PS C:\Users\DELL>
```

3.2 Gestion des Sessions et Compteurs

- **Sessions :** Stockage des métadonnées de connexion avec expiration automatique.
- **Temps réel :** Utilisation des `Sorted Sets (ZSET)` pour maintenir un classement dynamique des morceaux les plus écoutés.



PARTIE 4 : NEO4J - GRAPHE SOCIAL ET RECOMMANDATIONS

Objectif : Gérer les relations complexes et les algorithmes de recommandation.

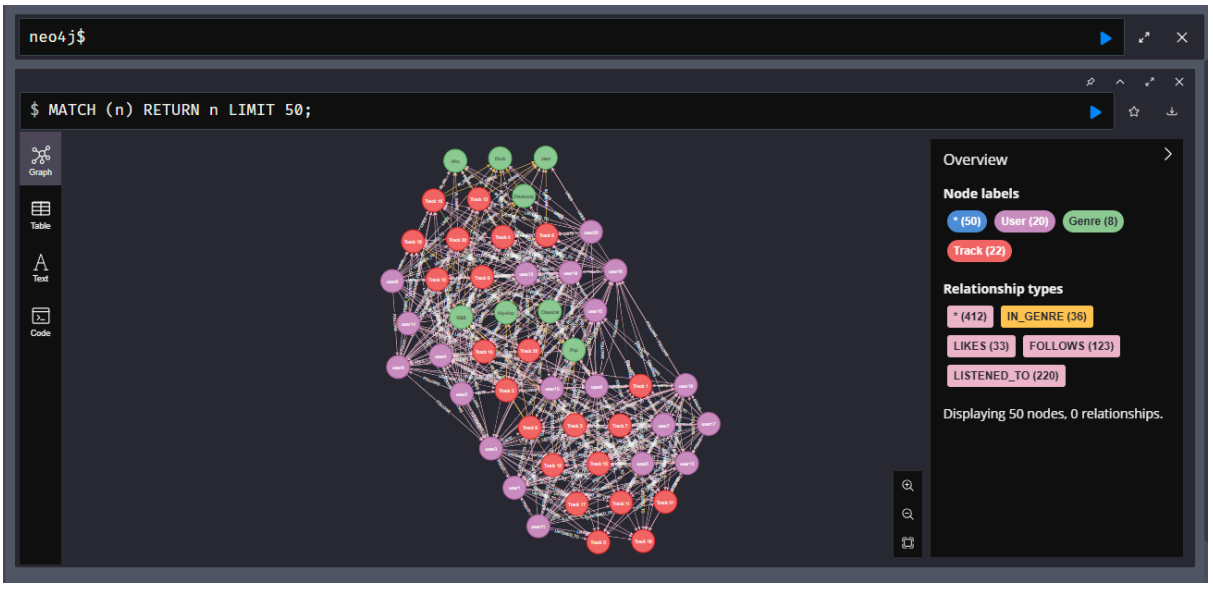
4.1 Modèle de Graphe

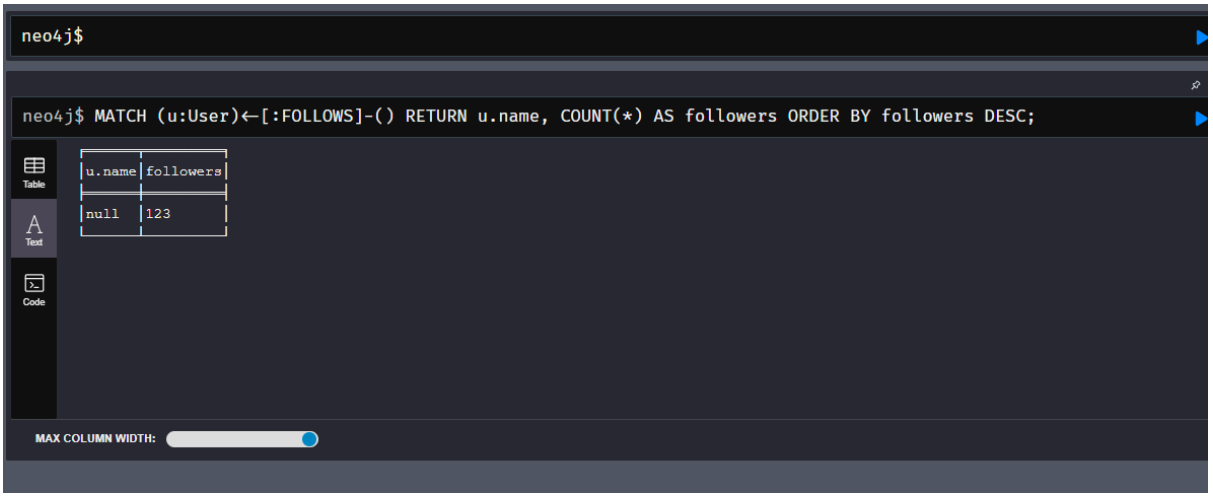
- **Nœuds :** `User`, `Track`, `Genre`.
- **Relations :** `FOLLOWS` (entre utilisateurs), `LISTENED_TO` (écoutes), `LIKES` (préférences).

4.2 Algorithmes de Recommandation

J'ai implémenté des requêtes Cypher permettant de :

1. Recommander des musiques écoutées par des utilisateurs ayant des goûts similaires.
2. Identifier les "influenceurs" (utilisateurs les plus suivis).
3. Calculer le chemin le plus court entre deux fans.





RÉFLEXION TECHNIQUE ET PERSPECTIVES

Défis rencontrés

Le principal défi a été la transition du modèle relationnel vers le modèle "colonnes larges" de Cassandra, qui impose de connaître ses requêtes avant de créer ses tables.

Perspectives

- **Évolution :** L'ajout de **Kafka** permettrait de synchroniser les écoutes entre Cassandra et Redis de manière asynchrone.
- **Amélioration :** Utilisation des index secondaires dans MongoDB pour affiner les recherches multicritères.

CONCLUSION

L'architecture proposée pour **SpotifYnov** démontre qu'en isolant les cas d'usage (catalogue, logs, social, cache), on obtient un système robuste, capable de supporter une montée en charge importante tout en offrant une expérience utilisateur fluide.